

ASSEMBLYAI VOICE AGENT HACKATHON

Voice Action Gate

An irreversible action runs only if **every argument** traces back to words you **actually said**.

Missing evidence is a refusal. There is **no code path** to “go ahead anyway.”

THE PROBLEM

One token apart.
A factor of **ten**.

transfer five hundred

transfer five thousand

Acoustically close. Ten times apart. This is why voice agents in banking, healthcare and insurance are still mostly **read-only** — happy to read you a balance, unwilling to move money.

WHY NOT JUST RAISE ACCURACY

Accuracy is an **asymptote**.

It never reaches a point where executing an **irreversible** action on a guess becomes acceptable. So this project does not try to make the failure rarer. **It changes what the failure does.**

A wrong block

The user repeats one sentence.

A wrong pass

Real money moves. Nobody gets to repeat anything.

The two directions are not symmetric — so the system is not built symmetric.

THE MECHANISM

A **capability**, not a check.

Code that performs an irreversible action does not accept arguments. It accepts an `Execute` credential — and that credential has **exactly one construction site**: the gate.

- The gate mints it only when **every argument** of the proposal is grounded in a witness drawn from the word-level transcript.
- “I could not tell” is a **refusal**, never a pass.

A check can be forgotten at one call site. A missing credential cannot be — there is no expression you could write that produces one.

THE LOAD-BEARING DETAIL

Witnesses are built **before** the proposal is visible.

The witness set is generated from the **transcript alone**. The component that produces it is structurally unable to see what the model proposed.

Otherwise a model that invents a number gets to pick its own search query — and an unrelated number said thirty seconds earlier becomes its evidence.

This is a **red line in the code layout**, not a convention: the proposal is not a parameter of the witness generator, so “just peek at it” is not an option a future edit can take.

THE SECOND DETAIL

The normalizer is a partial function on purpose.

Turning `five hundred` into `500` is allowed to answer **UNDECODABLE** — and UNDECODABLE is not a pass.

A normalizer with a fallback return value quietly converts “**I cannot read this**” into “**I believe it equals something**”. That is the failure direction that costs money.

Concretely: a bare “dollars” does not decode. It does not determine *which* dollar — so it earns no witness, and the transfer blocks. That behaviour is deliberate, and the demo shows it.

WHAT RUNS WHERE

Nothing sensitive passes through **our** server.

STAGE	WHERE	WHY THERE
Microphone → streaming transcript	Browser → AssemblyAI, direct	Audio never reaches us; the main API key never leaves the edge
Temporary session token	Worker (edge)	Short TTL, and it stamps the session cap
The gate	Worker, Python on Pyodide	Dependency-free; the same unit runs locally and at the edge
Daily issuance quota	Durable Object	Single-threaded, strongly consistent — a counter that is only eventually consistent cannot honestly be called fail-closed

What we actually ran.

CLAIM	STATUS	OBSERVATION
The gate runs unchanged on the edge runtime	measured	Whole suite executed inside the Worker : 151 / 152 pass
... the one that did not	did not run	A thread-safety property; the runtime is single-threaded, so it is inapplicable there — thread-safety evidence stays local-only
Session cap is enforced by AssemblyAI, not by our timer	measured	Server closes at <code>expires_at + ~60 s</code> , <code>error_code 3008</code> . The ~60 s is observed (n=2), not documented
The daily quota has teeth	measured	With limit = 2, three requests give 502 / 502 / 429 ; the counter stops at 2
End-to-end with a live microphone	not yet run	Token minting through the Worker is verified; the full mic → transcript → gate path needs a real microphone

A PUBLIC URL THAT BILLS BY THE SECOND

The cost boundary is an equation, not a promise.

worst case per connection = cap + ~60 s

worst case per day = tokens issued × (cap + ~60 s)

Both factors are set by our Worker: it issues the token and it writes the cap. The **enforcement** is on AssemblyAI's side, which is why it cannot be bypassed by editing the frontend — the connection is not held by the frontend.

The ~60 s is an observed lag, not a documented guarantee. Stated that way on purpose.

THREAT MODEL — STATED, NOT IMPLIED

What this does **not** defend against.

- A **malicious client**. The gate consumes a transcript the client supplies, so a client can forge one — but the adversary here is the **language model**, and the person at the microphone is the party being protected. Forging your own transcript is lying to yourself.
- A wrong transcript that is **internally consistent**. If the recognizer confidently mishears, and the model proposes exactly what was misheard, the arguments are grounded and the action passes. The gate proves **provenance**, not truth.

Both are written into the architecture document, because “our threat model excludes this” and “we did not think of it” look identical in a demo. Only one of them is written down in advance.

IN ONE LINE

**The failure mode of a voice agent
should be “say that again”,
not a wire transfer.**

A dependency-free Python unit you can drop into someone else's agent. The tests pin both properties — including a mutation arm that turns red if the structural separation is undone.